

```
basicstyle=, breaklines=true, breakatwhitespace=true, frame=single, backgroundcolor=,
keywordstyle=, stringstyle=, commentstyle=, showstringspaces=false, language=Python
```

Lab 1: DDA & Bresenham Line Drawing Algorithms

Computer Graphics Laboratory

December 7, 2025

Overview

This lab explores two fundamental line rasterization algorithms in computer graphics: the **Digital Differential Analyzer (DDA)** and **Bresenham's algorithm**. Both convert continuous geometric lines into discrete pixels for display on raster displays.

1 Objectives

- **Understand DDA:** Implement the Digital Differential Analyzer algorithm for line rasterization using floating-point slope calculations.
- **Understand Bresenham:** Implement Bresenham's integer-only line algorithm that eliminates floating-point arithmetic for improved efficiency.
- **Compare methods:** Analyze trade-offs in precision, performance, and handling of edge cases (e.g., vertical/horizontal lines, various slopes).

2 Algorithm Description

2.1 DDA Line Drawing

The Digital Differential Analyzer

The DDA algorithm rasterizes a line segment by computing pixel coordinates based on the line's slope:

1. **Calculate differences:** $\Delta x = x_2 - x_1$, $\Delta y = y_2 - y_1$
2. **Compute slope:** $m = \frac{\Delta y}{\Delta x}$ (special handling when $\Delta x = 0$ for vertical lines)
3. **Iterate over x:** For each integer x from x_1 to x_2 , compute $y = y_1 + m(x - x_1)$
4. **Rasterize:** Plot pixel at $(x, \text{round}(y))$

Advantage: Simple and intuitive. *Disadvantage:* Uses floating-point division and multiplication; rounding errors can accumulate.

2.2 Bresenham Line Drawing

The Integer-Only Algorithm

Bresenham's algorithm achieves line rasterization using **only integer arithmetic**, making it faster and avoiding rounding errors:

1. **Compute absolute differences:** $dx = |x_2 - x_1|$, $dy = |y_2 - y_1|$
2. **Set step directions:** $x_step, y_step \in \{-1, +1\}$ based on line direction
3. **Initialize decision parameter:** p used to decide which pixel to choose next
4. **Two cases:**
 - **If** $dx > dy$ (shallow slope): iterate over x , incrementally update p
 - **If** $dy \geq dx$ (steep slope): iterate over y , incrementally update p

Advantage: Integer-only, highly efficient. *Disadvantage:* More complex logic; branch for slope handling.

3 Code Implementation

3.1 DDA Algorithm

Python implementation from notebook:

```
def DDA_Line(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    m = dy / dx if dx != 0 else None
    x_points, y_points = [], []
    if dx != 0:
        step = 1 if x2 > x1 else -1
        for x in range(x1, x2 + step, step):
            y = y1 + m * (x - x1)
            x_points.append(x); y_points.append(round(y))
    else:
        step = 1 if y2 > y1 else -1
        for y in range(y1, y2 + step, step):
            x_points.append(x1); y_points.append(y)
```

3.2 Bresenham Algorithm

Python implementation from notebook:

```
def bresenham_line(x1, y1, x2, y2):
    dx, dy = abs(x2 - x1), abs(y2 - y1)
    x_step = 1 if x1 < x2 else -1
    y_step = 1 if y1 < y2 else -1
    x, y = x1, y1
    x_points, y_points = [x], [y]
    if dx > dy:
        p = 2 * dy - dx
        for _ in range(dx):
            x += x_step
            if p < 0: p += 2 * dy
            else: y += y_step; p += 2*dy - 2*dx
            x_points.append(x); y_points.append(y)
    else:
        p = 2 * dx - dy
        for _ in range(dy):
            y += y_step
            if p < 0: p += 2 * dx
            else: x += x_step; p += 2*dx - 2*dy
            x_points.append(x); y_points.append(y)
```

Output

Example Case: Line from $(x_1, y_1) = (2, 3)$ to $(x_2, y_2) = (10, 7)$

DDA Output: Green markers with smooth rasterization following the line's slope $m = \frac{4}{8} = 0.5$. Pixel coordinates computed using floating-point arithmetic and rounded.

Bresenham Output: Blue markers with crisp integer-only rasterization. No floating-point errors or cumulative rounding drift.

Both algorithms produce visually similar results, but Bresenham is more efficient and precise for real-time graphics applications.

Discussion

1. Precision and Accuracy

- **DDA:** Uses floating-point arithmetic; subject to rounding errors and cumulative drift over long lines.
- **Bresenham:** Integer-only arithmetic eliminates floating-point errors; rasterization is exact and reproducible.

2. Performance

- **DDA:** Requires division (to compute slope) and multiplication at each

iteration—expensive.

- **Bresenham:** Only addition and bit shifts (powers of 2); highly optimized for hardware implementation.

3. Slope Handling

- **DDA:** Single unified loop; naturally handles all slopes but with floating-point overhead.
- **Bresenham:** Branches into two cases ($|m| < 1$ vs. $|m| \geq 1$) for optimal integer decisions; more complex logic.

4. Generalization

- The incremental-error concept used by Bresenham extends beautifully to circle drawing, as demonstrated in the Bresenham Circle algorithm in the notebook.
- Modern GPUs still employ variants of Bresenham for anti-aliased rasterization.

5. Conclusion

- For teaching and understanding rasterization, DDA's simplicity is pedagogically valuable.
- For production systems and real-time graphics, Bresenham's efficiency, accuracy, and integer-only design make it superior.